Guía de pruebas y capturas de la actividad autónoma de PharmaSoft

Actividad autónoma de la sesión 13

Estudiante: Brandon Erly Pacho Mamani

Curso: Lenguaje de Programación II

Proyectos: PharmaBackend y pharma-frontend

Rama: feature/optimizacion-produccion-pacho

Esta guía explica cómo repetir las comprobaciones de las partes A a F y reunir las capturas del informe. Cada ficha indica el lugar de trabajo, la preparación, el procedimiento, el resultado esperado y lo que debe verse en cada imagen. Los resultados observados se completan con tu ejecución; una instrucción o una prueba unitaria aprobada no sustituye la evidencia del navegador, Redis o el API.

Las 14 pruebas de la práctica ya están en tu documento de la sesión 13. Conserva sus capturas y reutilízalas cuando acrediten exactamente el mismo requisito. Para la mejora adicional del Reto 02, la referencia es el final de la práctica, no el proyecto anterior a esa práctica. La guía usa fichas y tablas del documento editable de S13 y los campos de evidencia de las autónomas S10 y S11.

Cómo organizar esta ronda

1. Crea en PharmaBackend una carpeta `docs/evidencias-s13-autonoma-manual/AAAA-MM-DD/`. Guarda allí las capturas de esta ronda. Puedes copiarla al frontend al terminar. No reemplaces `evidencias-s13` de la práctica ni las mediciones automatizadas anteriores.
2. Usa el prefijo `AA-` de esta guía: identifica la actividad autónoma y evita confundirla con las capturas `P01`, `P02` de la práctica.
3. Mantén una ficha por prueba con estos campos: fecha, configuración, pasos, esperado, observado, estado, archivos de evidencia e interpretación. Estado inicial: **Pendiente de tu ejecución**.
4. Estados válidos: **Conforme**, **No conforme** o **Evidencia incompleta**. No marques Conforme si solo tienes parte del antes y después.
5. Usa capturas originales de pantalla. En navegador muestra URL, solicitud y resultado. En Postman muestra método, URL, estado y cuerpo pertinente. Para cabeceras, captura **Response Headers**, no el encabezado Authorization de la petición.
6. Los nombres `main-*.js` y `chunk-*.js` cambian al compilar. Usa los nombres de tu ejecución. No pegues como propios los valores anteriores de 313,59 o 305,78 kB sin su captura y contexto.
7. Antes de compartir una imagen, oculta tokens completos, contraseñas, secreto JWT y DNI. Nunca adjuntes un HAR sin revisar: puede incluir autorización y datos del login. Ocultar la respuesta con CSS o recortar una pantalla no modifica los datos guardados en un HAR.
8. Para el informe, coloca cada imagen debajo de su prueba, con número de figura y una frase que explique qué demuestra. Usa un ancho máximo de 17 cm, conserva la proporción y divide una captura si su texto resulta ilegible. Una tabla recreada con datos sirve de resumen; no reemplaza la captura original que respalda la cifra.

Los nombres propuestos son un inventario de evidencias, no pruebas adicionales. Puedes agrupar varias cabeceras o reutilizar una captura cuando demuestre ambos requisitos y siga siendo legible. Los tres errores adicionales de D02 y la suite de F01 son complementarios. Lo obligatorio es acreditar todas las condiciones de la consigna, no producir imágenes duplicadas.

Preparación común

Carpetas de trabajo. Cuando se indique FRONTEND, abre una terminal en `pharma-frontend`. Cuando se indique BACKEND, ábrela en `PharmaBackend`. No mezcles ambos directorios al ejecutar comandos.

1. Abre ambos proyectos y revisa la rama con `git branch --show-current`. Conserva los cambios que ya tienes; no uses reset ni descartes archivos para repetir una medición.

2. Inicia Oracle desde Docker Desktop. En este equipo el servicio local de Oracle usa el puerto del anfitrión 1522. No cambies las credenciales ni el puerto de conexión que ya funcionan.

3. En FRONTEND ejecuta los dos comandos siguientes, uno por uno. El segundo inicia únicamente Nginx y Redis del compose de la práctica.

```
npm.cmd run build -- --stats-json --verbose
docker compose -f deploy/docker-compose.yml up -d
```

4. Inicia PharmaBackend con la configuración de producción local que ya utilizas. En IntelliJ, comprueba `SPRING_PROFILES_ACTIVE=prod`, Oracle y el secreto JWT en las variables de entorno de la configuración de ejecución. No captures sus valores. Para el trabajo normal deja `CACHE_TTL=10m`. El perfil prod no muestra Swagger; usa Postman o el navegador.

5. No ejecutes dos instancias en 8080. Si arrancas un JAR, detén esa instancia antes de reconstruirlo; en Windows el archivo puede quedar bloqueado mientras se ejecuta. Elige IntelliJ o JAR para la ronda y anótalo.

6. Abre `http://localhost:8080/actuator/health` sin token. Con las dependencias disponibles debe mostrar `UP`, normalmente HTTP 200, sin los detalles de los componentes.

7. Abre Chrome en `http://localhost:8081`. Usa esta SPA de Nginx para la autónoma. El puerto 4200 no permite comprobar la política de caché de Nginx.

8. En Postman crea una colección local para la autónoma con `apiUrl=http://localhost:8080/api/v1` y `backendUrl=http://localhost:8080`. Inicia sesión con tu cuenta ADMIN en `POST {{apiUrl}}/auth/login`. Guarda el token solo en una variable local de tipo secreto o en el almacén local de Postman; configura Bearer Token en la colección. No captures el cuerpo del login con el JWT ni exportes su valor.

9. Crea una petición separada llamada «Sin autenticación» y configúrala como **No Auth**. Servirá para health público y errores 401. Las métricas y los detalles de health deben consultarse con el ADMIN.

10. Prepara datos de ensayo propios. Usa una marca nueva, por ejemplo `ZZ S13 AA ronda01`, y anota los IDs reales: una categoría con producto, una categoría vacía, un producto de precio 2,50 y stock 10 y un cliente ficticio activo. Puedes crearlos en la SPA. No uses productos de venta real para cambiar precios o stock. No publiques el DNI del cliente, aunque sea de ensayo.

11. Cierra otras pestañas de la SPA antes de medir Redis. El listado de categorías de la SPA pide `soloActivas=true`; el GET sin ese parámetro conserva la consulta administrativa. Son claves distintas. No cambies URL, tamaño, orden ni usuario en medio de una serie.

12. Abre DevTools con F12. Deja visibles las columnas **Name**, **Status**, **Type**, **Size** y **Time**. Desactiva cualquier filtro de texto que pudiera ocultar solicitudes relevantes.

Capturas de preparación. `AA-00-01-servicios.png`: Nginx, Redis y Oracle en ejecución, sin paneles de variables privadas. `AA-00-02-health-inicial.png`: URL de health, HTTP 200 y estado UP sin token. No hace falta otra captura del login.

Orden recomendado. A01 → A02 → A03 → B01 → B02 → B03 → C01 → C02 → C03 → C04 → C05 → C06 → C07 y D01 con la misma venta → C08 → C09 → D02 → D03 → D04 → E01 → F01 → F02. Cambia TTL y precarga solo en sus pruebas y restáuralos al terminar. C09 reúne también las cuatro comprobaciones de health.

A01 Analizador y cinco módulos iniciales

Requisito: parte A, analizar browser-stats.json e identificar los cinco módulos más pesados del paquete inicial y su origen. **Lugar:** FRONTEND, terminal, explorador y analizador de esbuild.

1. Con la configuración de producción final, ejecuta `npm.cmd run build -- --stats-json --verbose` y espera el mensaje de compilación terminada.
2. Captura **Initial chunk files**, **Initial total**, **Lazy chunk files** y el final del build. Añade una captura si la lista no cabe; no ocultes una advertencia de presupuesto.
3. Localiza `dist/pharma-frontend/browser-stats.json`. Guarda una copia con fecha en la carpeta de mediciones de esta ronda antes de volver a compilar.
4. Abre `https://esbuild.github.io/analyze/`, pulsa **Import your metafile** y selecciona ese archivo. Es el metafile, no un JS ni un JSON de otra compilación.
5. Revisa el treemap. Identifica los archivos iniciales usando la lista del build: main y sus chunks compartidos, además de styles. No confundas el total de todos los outputs del analizador con Initial total.
6. Amplía los chunks iniciales y registra los cinco archivos fuente con mayor aporte. Si un módulo aparece en varios outputs iniciales, suma sus aportes. Anota la unidad del analizador: si indica KiB o unidades de 1024, conviértelas antes de compararlas con kB decimales del CLI.
7. Completa una tabla: módulo o archivo fuente, bytes aportados al inicial, origen Angular/RxJS/código propio/tercero, y dependencia o función. Que el nombre de un archivo contenga «debug» no demuestra que se haya compilado en desarrollo.

Esperado: el metafile abre y la tabla describe cinco módulos del grafo inicial, con cifras respaldadas por el analizador. Los chunks de ventas o productos diferidos no se suman al inicial por estar en el mismo gráfico.

Capturas necesarias. `AA-A01-01-build-initial.png`: comando y tamaño inicial. `AA-A01-02-build-lazy.png`: lista diferida y compilación terminada. `AA-A01-03-analyzer-general.png`: treemap con el archivo importado. `AA-A01-04-modulos-iniciales.png`: detalle legible de los módulos; usa sufijos a, b o c si necesitas varias vistas.

Registrar: versión de Node y Angular, configuración de build, nombre del metafile, cinco aportes y explicación.
Evidencia incompleta si: solo se entrega una tabla digitada o el total global del analizador.

Resultado observado: Pendiente de tu ejecución. Registrar aquí cifras, IDs y lo que realmente ocurrió.

Estado: Pendiente. Cambiar a Conforme, No conforme o Evidencia incompleta al revisar las capturas.

Evidencias adjuntas: Anotar los nombres de los archivos que respaldan esta ficha.

A02 Mejora adicional y tamaño antes y después

Requisito: parte A, demostrar una mejora adicional del tamaño de entrega. **Lugar:** FRONTEND, código y terminal.

1. Conserva como «antes del Reto 02» la compilación final de la práctica que documenta MainLayout cargado desde el inicio. En el informe práctico se registraron 313,59 kB. Busca su captura original y confirma versión, build de producción y dependencias; no uses el valor de 303,64 kB anterior a la práctica para atribuir toda la diferencia a esta mejora.
2. Si falta una línea base comparable, no la reconstruyas descartando cambios actuales. Repite el estado anterior en una copia de trabajo separada o deja la evidencia anterior pendiente hasta revisarla. Mantén las dependencias y las opciones de compilación iguales.
3. En `src/app/app.routes.ts` comprueba la mejora ya incorporada: la ruta protegida utiliza `loadComponent` para MainLayout y conserva `authGuard`. El estado anterior lo importaba estáticamente y usaba `component: MainLayout`.
4. Captura la comparación de ese cambio, sin otros archivos ni variables privadas en pantalla. No necesitas volver a escribir la mejora si ya está aplicada.
5. Ejecuta otra compilación de producción con `--stats-json --verbose`. Captura Initial total, el chunk diferido de MainLayout y el final correcto. Guarda el metafile como «después».
6. Calcula reducción en bytes = antes − después y reducción porcentual = (antes − después) / antes × 100. Compara Raw size con Raw size. Presenta Estimated transfer size en una columna separada.
7. Abre la SPA: verifica login, Inicio, Productos y Ventas. Una reducción que impida navegar no es una mejora aceptable.

Esperado: MainLayout se descarga bajo demanda y el paquete inicial disminuye con un método comparable. Los resultados nuevos pueden diferir de los 305,78 kB registrados anteriormente; anota lo que obtengas.

Capturas necesarias. `AA-A02-01-antes.png`: build original comparable. `AA-A02-02-cambio-layout.png`: cambio de ruta. `AA-A02-03-despues.png`: Initial total, chunk MainLayout y compilación correcta. La captura de A01 puede reutilizarse si corresponde exactamente a ese build.

Registrar: versiones de ambos estados, tamaños, diferencia, porcentaje y comprobación de navegación. No sustituir la captura del antes por un número transcrito.

Resultado observado: Pendiente de tu ejecución. Registrar aquí cifras, IDs y lo que realmente ocurrió.

Estado: Pendiente. Cambiar a Conforme, No conforme o Evidencia incompleta al revisar las capturas.

Evidencias adjuntas: Anotar los nombres de los archivos que respaldan esta ficha.

A03 Navegación con y sin precarga

Requisito: parte A, comparar bytes de la primera pantalla y tiempo hasta ver Ventas tras el login. **Lugar:** FRONTEND, Nginx y Chrome.

1. Conserva una copia del contenido actual de `src/app/app.config.ts`. La variante sin precarga usa `provideRouter(routes, withComponentInputBinding())`.
2. Compila esa variante en producción. No midas mientras el build todavía escribe en dist.
3. Cierra todas las ventanas de incógnito y abre una nueva. Entra en `/login`, abre F12, activa **Disable cache** y **Preserve log** y selecciona una condición de red fija. Usa la misma condición en las dos variantes; registra si es Sin limitación o una conexión limitada. No cambies de equipo ni agregues otras descargas entre variantes.
4. Para medir el intervalo de navegación puedes usar el ayudante de consola del anexo 1. Instálalo antes de pulsar Ingresar. Solo observa botones y el encabezado de Ventas; no lee credenciales.
5. Limpia Red una vez antes del login. Inicia sesión como ADMIN, espera exactamente 5 segundos en Inicio y captura el total **Transferred** de **All**, sin filtro. Este es el total de la ventana login a primera pantalla; incluye los recursos y las peticiones realizadas hasta ese momento.
6. Abre Ventas desde el menú. Guarda los tiempos del ayudante: login a Ventas y clic del menú a Ventas. Ambos se miden hasta el encabezado visible, no hasta completar cada petición del listado. Si eliges medir datos completamente cargados, usa ese mismo criterio en las dos variantes y decláralo.
7. Repite cinco veces con contextos nuevos. Conserva las cinco filas en una tabla; calcula medias. No elijas solo la ejecución más rápida. Si usas cronómetro manual, declara que el tiempo es aproximado y no lo presentes con precisión de milisegundos.
8. Para la variante con precarga, añade `withPreloading` y `PreloadAllModules` al import de `@angular/router` y configura el provider como se muestra abajo. Compila y repite los pasos 3 a 7 con la misma red, usuario y espera.

```
provideRouter(routes, withComponentInputBinding(), withPreloading(PreloadAllModules))
```

9. Compara medias de bytes, tiempo login a Ventas y tiempo clic a Ventas. Decide qué conviene para los tres locales y justifica con esa diferencia. El tiempo login a Ventas incluye la espera fija de 5 segundos.
10. Restaura el contenido original de `app.config.ts` y vuelve a compilar. Captura la configuración final elegida. No dejes la variante experimental activada por olvido.

Esperado: una comparación reproducible; la precarga puede adelantar descargas y acortar el clic a Ventas a costa de bytes iniciales. No se exige obtener exactamente las cifras del informe anterior.

Capturas necesarias. `AA-A03-01-sin-config.png`, `AA-A03-02-sin-red.png`, `AA-A03-03-sin-tiempos.png`, `AA-A03-04-con-config.png`, `AA-A03-05-con-red.png`, `AA-A03-06-con-tiempos.png` y `AA-A03-07-restauracion.png`. Numera las capturas de las cinco repeticiones o conserva el registro legible de todas; una captura de una sola corrida no respalda una media de cinco.

Resultado observado: Pendiente de tu ejecución. Registrar aquí cifras, IDs y lo que realmente ocurrió.

Estado: Pendiente. Cambiar a Conforme, No conforme o Evidencia incompleta al revisar las capturas.

Evidencias adjuntas: Anotar los nombres de los archivos que respaldan esta ficha.

B01 Encabezados de los cinco recursos

Requisito: parte B, comprobar index, main, chunk de ventas, styles y categorías. **Lugar:** Chrome en 8081, terminal FRONTEND y Postman.

1. Identifica los archivos reales de la compilación final. Para el chunk de ventas, entra en Ventas y revisa el JS nuevo en Red; también puedes relacionarlo con `ventas-routes` en el build o con su entryPoint en el metafile.

2. En terminal ejecuta por separado `curl.exe -I` sobre index.html, main, el chunk de ventas y styles. Sustituye los ejemplos por nombres existentes.

```
curl.exe -I http://localhost:8081/index.html
curl.exe -I http://localhost:8081/main-NOMBRE-REAL.js
curl.exe -I http://localhost:8081/chunk-NOMBRE-VENTAS.js
curl.exe -I http://localhost:8081/styles-NOMBRE-REAL.css
```

3. Para categorías usa el mismo origen de Nginx, `/api/v1/categorias`, con ADMIN. Para acreditar curl sin pegar un JWT en un comando usa el ayudante PowerShell del anexo 2, que lo entrega a curl por entrada estándar. No uses Copy as cURL en una terminal cuyo historial pueda conservar la autorización completa.

4. En Chrome puedes activar Disable cache solo para obtener cabeceras desde red. Abre `http://localhost:8081/index.html` con Preserve log, selecciona la primera solicitud de tipo Doc y mira **Response Headers**. Repite para los JS, CSS y el GET autenticado de categorías.

5. En categorías comprueba el cuerpo del GET además de la cabecera: un 401 sin token no demuestra que hayas ensayado una respuesta autenticada válida.

6. Completa la tabla de cinco recursos con su Cache-Control exacto. La columna de origen de recarga se obtiene en B02; no la deduzcas aquí a partir del encabezado.

Esperado: HTML con `no-cache`; JS/CSS con hash, `public, max-age=31536000, immutable`; API con `no-cache, no-store, max-age=0, must-revalidate`. Anota también Pragma y Expires cuando aparezcan. No-store restringe el almacenamiento de la respuesta del API aunque el servidor utilice Redis.

Capturas necesarias. `AA-B01-01-curl-estaticos.png` y `AA-B01-02-curl-api.png`: URL o etiqueta del recurso, HTTP y Cache-Control. `AA-B01-03-index.png`, `AA-B01-04-main.png`, `AA-B01-05-ventas.png`, `AA-B01-06-styles.png` y `AA-B01-07-api.png`: cabeceras de respuesta de los cinco recursos en Red. Si agrupas recursos, cada encabezado debe seguir legible.

Resultado observado: Pendiente de tu ejecución. Registrar aquí cifras, IDs y lo que realmente ocurrió.

Estado: Pendiente. Cambiar a Conforme, No conforme o Evidencia incompleta al revisar las capturas.

Evidencias adjuntas: Anotar los nombres de los archivos que respaldan esta ficha.

B02 Origen de los recursos en una recarga normal

Requisito: parte B, registrar red, memoria, disco o 304. **Lugar:** Chrome, SPA por Nginx.

1. Inicia sesión y abre Ventas para que se descargue su chunk. Para incluir categorías en la misma ronda, visita también Categorías y conserva los nombres de sus solicitudes.
2. En Red selecciona **All**, quita el texto del filtro, desmarca **Disable cache**, desactiva Preserve log y deja la misma red de la ronda. Es indispensable permitir la caché para esta comprobación.
3. Para establecer la primera carga puedes vaciar caché y recargar de forma forzada una sola vez. Espera que termine y captura el total Transferred y las columnas Status y Size.
4. Pulsa F5 normal, sin Ctrl+F5 ni otra limpieza. Captura el total transferido, JS/CSS y el documento. Si el chunk de ventas no aparece, comprueba que estás en Ventas y no únicamente en Inicio.
5. Abre Categorías y repite F5 normal para observar su GET. La respuesta del API debe ir a red; guarda su 200 y la cabecera no-store.
6. Completa la tabla de B01 usando lo observado en cada recurso. Un 200 mostrado junto a «memory cache» no es una descarga de red. Un 304 corresponde a una revalidación, no a un cuerpo completo transferido. Si index devolvió 200 en vez de 304, anota 200/red sin corregir artificialmente el dato.

Capturas necesarias. `AA-B02-01-primera-carga.png`, `AA-B02-02-f5-ventas.png` y `AA-B02-03-f5-categorias.png`: URL, Disable cache desmarcado, Status, Size y total Transferred. Añade `AA-B02-04-documento.png` si necesitas acreditar el 200/304 del HTML.

Registrar: los cinco orígenes reales y la diferencia de bytes. No comparar «resources» descomprimidos de una corrida con «transferred» de otra.

Resultado observado: Pendiente de tu ejecución. Registrar aquí cifras, IDs y lo que realmente ocurrió.

Estado: Pendiente. Cambiar a Conforme, No conforme o Evidencia incompleta al revisar las capturas.

Evidencias adjuntas: Anotar los nombres de los archivos que respaldan esta ficha.

B03 Despliegue sin detener Nginx

Requisito: parte B, cambiar un texto de Productos, reconstruir y explicar la actualización sin borrar caché. **Lugar:** FRONTEND, Chrome y Docker Desktop.

1. Abre Productos con una sesión válida y Disable cache desmarcado. Conserva el mismo navegador durante toda la prueba.
2. Captura el texto visible actual y los nombres de main y del chunk de Productos en Red. Guarda también una lista de los JS/CSS actuales de `dist/pharma-frontend/browser`.
3. En `producto-list.html` cambia únicamente una palabra visible del título, por ejemplo añade «versión de prueba 2». No vuelvas a aplicar el texto del ensayo anterior como si fuera un cambio nuevo.
4. Ejecuta el build de producción con stats-json. No detengas Nginx, Redis ni la sesión del navegador. Espera al final correcto del build.
5. Pulsa F5 normal en la pestaña que mantuviste abierta, sin borrar caché. Captura el título nuevo y Red.
6. Compara qué nombres cambiaron y qué archivos conservan hash. Selecciona un archivo nuevo y otro reutilizado para mostrar sus orígenes. No afirmes que cambió solo un chunk: los imports pueden propagar nuevos hashes a main y otros archivos.
7. Explica que el HTML revalidado referencia la nueva cadena de archivos; las URLs con hashes conservados pueden seguir en caché. El hash final de tu ejecución puede ser distinto del anterior.
8. Deja el texto final que decidas conservar y vuelve a compilar si lo cambias de nuevo. Documenta esa restauración. El montaje local de dist no equivale a un despliegue atómico apto para producción.

Capturas necesarias. `AA-B03-01-antes.png`: título y nombres anteriores. `AA-B03-02-build.png`: reconstrucción correcta. `AA-B03-03-despues.png`: título nuevo y recarga del mismo contexto. `AA-B03-04-archivos.png`: archivos cambiados y conservados, con su origen en Red. `AA-B03-05-nginx.png`: Nginx sigue ejecutándose; reutiliza la vista de Docker si acredita la continuidad de esta ronda.

Resultado observado: Pendiente de tu ejecución. Registrar aquí cifras, IDs y lo que realmente ocurrió.

Estado: Pendiente. Cambiar a Conforme, No conforme o Evidencia incompleta al revisar las capturas.

Evidencias adjuntas: Anotar los nombres de los archivos que respaldan esta ficha.

C01 Veinte consultas de categorías y tasa de aciertos

Requisito: parte C, aciertos, fallos y tiempos de categorías. **Lugar:** BACKEND, Postman y terminal FRONTEND para Redis.

1. Cierra las pestañas de la SPA que solicitan categorías y reinicia solo el backend con CACHE_TTL=10m. Renueva la autenticación de Postman si cambió el secreto del arranque.

2. Usa exactamente `GET {{apiUrl}}/categorias?pagina=0&tamanio=10&ordenarPor=nombre&direccion=asc&soloActivas=false` durante las 20 consultas.

3. Desde FRONTEND inspecciona las claves, sin consultar el GET todavía:

```
docker compose -f deploy/docker-compose.yml exec -T redis redis-cli --scan --pattern 'categorias::*'
```

4. Para esta URL la clave actual es `categorias::pagina-0-10-nombre-asc-activas-false`. Si ya existe, elimina únicamente esa clave del entorno de ensayo con el comando siguiente. Un reinicio de Java no borra Redis. No uses FLUSHALL ni borres claves de otros trabajos.

```
docker compose -f deploy/docker-compose.yml exec -T redis redis-cli DEL 'categorias::pagina-0-10-nombre-asc-activas-false'
```

5. Antes de los GET consulta y captura ambos contadores con ADMIN. Si la métrica aún no existe, anota ese estado sin emitir un GET de categorías; asegúrate de que las cachés están predeclaradas y las estadísticas activadas en application.yaml.

```
GET {{backendUrl}}/actuator/metrics/cache.gets?tag=name:categorias&tag=result:hit
GET {{backendUrl}}/actuator/metrics/cache.gets?tag=name:categorias&tag=result:miss
```

6. En Postman, ejecuta la petición de categorías una sola vez. Captura el 200, URL completa, contenido y tiempo total. Esta es la consulta 1 y debe ser el fallo con la clave inicialmente ausente.

7. Ejecuta otras 19 veces la misma petición con el Runner, sin incluir login ni otra lectura de categorías en esa colección de ejecución. Conserva los 19 tiempos individuales. Entre la consulta 1 y la 20 no crees ni edites categorías y evita que transcurran 10 minutos.

8. Consulta de nuevo hit y miss. Calcula deltas: H = hit después − hit antes; M = miss después − miss antes. Tasa = H / (H + M) × 100. No uses los contadores acumulados de toda la sesión como si fueran la muestra de 20.

9. Calcula la media de las consultas 2 a 20 y compárala con la consulta 1. El primer fallo tiene muestra n=1; no constituye una media general de fallos. Usa el mismo tiempo total de Postman para ambos grupos, sin compararlo con TTFB de Chrome.

Esperado en aislamiento: 1 fallo y 19 aciertos, tasa 95 %. Si no coincide, anota la cifra real y revisa parámetros, otras pestañas, expiración o invalidaciones. El tiempo incluye seguridad y transporte; un acierto no garantiza ser más rápido en todas las corridas.

Capturas necesarias. `AA-C01-01-clave-ausente.png`; `AA-C01-02-contadores-antes.png` (hit y miss legibles); `AA-C01-03-primer-get.png`; `AA-C01-04-runner-19.png`; `AA-C01-05-contadores-despues.png`. Guarda el registro de tiempos sin Authorization ni respuesta de login. La captura de Runner debe acreditar 19 ejecuciones, no una sola.

Resultado observado: Pendiente de tu ejecución. Registrar aquí cifras, IDs y lo que realmente ocurrió.

Estado: Pendiente. Cambiar a Conforme, No conforme o Evidencia incompleta al revisar las capturas.

Evidencias adjuntas: Anotar los nombres de los archivos que respaldan esta ficha.

C02 Veinte consultas de productos y tasa de aciertos

Requisito: parte C, repetir el análisis para productos. **Lugar:** los mismos de C01.

1. Conserva el método de C01 y cambia únicamente el catálogo. Usa `GET {{apiUrl}}/productos?pagina=0&tamanio=10&ordenarPor=nombre&direccion=asc`.
2. Inspecciona `productos::*` y elimina solo la clave `productos::pagina-0-10-nombre-asc` si existe. No borres categorías ni reinicies entre el primer GET y los 19 siguientes.
3. Registra contadores iniciales mediante cache.gets con `tag=name:productos` y result hit/miss.
4. Ejecuta una primera consulta y luego 19 idénticas. Durante la serie no registres ventas ni edites productos: esas operaciones invalidan la caché y cambiarían la condición del ensayo.
5. Registra contadores finales, calcula sus deltas y la tasa. Calcula la media de los tiempos 2 a 20 y compárala con el primero, manteniendo la misma herramienta y métrica.
6. Presenta ambas cachés en una tabla: H, M, tasa, primer GET, media de aciertos, número de observaciones y condiciones. Explica una desviación en vez de sustituirla por 95 %.

Capturas necesarias. `AA-C02-01-clave-ausente.png`, `AA-C02-02-contadores-antes.png`, `AA-C02-03-primer-get.png`, `AA-C02-04-runner-19.png` y `AA-C02-05-contadores-despues.png`. Deben mostrar productos, no las respuestas de métricas de categorías.

Resultado observado: Pendiente de tu ejecución. Registrar aquí cifras, IDs y lo que realmente ocurrió.

Estado: Pendiente. Cambiar a Conforme, No conforme o Evidencia incompleta al revisar las capturas.

Evidencias adjuntas: Anotar los nombres de los archivos que respaldan esta ficha.

C03 Vigencia de un minuto y restauración a diez

Requisito: parte C, demostrar expiración real mediante redis-cli. **Lugar:** configuración de ejecución del BACKEND, Postman y terminal FRONTEND.

1. Detén la instancia de backend con su propio control de parada. En las variables de la configuración de ejecución cambia solo `CACHE_TTL` de `10m` a `1m`. Captura únicamente esa variable, sin abrir valores privados de DB o JWT.
2. Inicia esa misma instancia y renueva login si hace falta. Usa una URL distinta de la de C01 para aislar la prueba: categorías con pagina=0, tamanio=7, ordenarPor=nombre, direccion=asc y soloActivas=false.
3. Inspecciona y elimina solo esa clave si existe, antes del GET. Luego realiza un único GET y no vuelvas a leerlo hasta comprobar la expiración.
4. Ejecuta el TTL inmediatamente y anota la hora. Debe ser un número positivo cercano a 60; si ya transcurrieron segundos, no tiene que ser exactamente 60.

```
Get-Date -Format o
docker compose -f deploy/docker-compose.yml exec -T redis redis-cli TTL 'categorias::pagina-0-7-nombre-asc-activas-false'
```

5. Espera al menos 65 segundos sin abrir Categorías ni repetir ese GET. Consulta el TTL de nuevo y captura la hora: `-2` significa que la clave no existe. `-1` significa clave sin expiración y no satisface el criterio. No uses EXPIRE para fabricar esta prueba.
6. Detén el backend, restaura `CACHE_TTL=10m` y reinicia. Realiza el mismo GET de la clave ya expirada y consulta TTL: debe quedar cerca de 600.
7. Captura la restauración y deja el sistema funcionando con 10 minutos. Cambiar el valor de configuración no modifica automáticamente el TTL de claves ya existentes.

Capturas necesarias. `AA-C03-01-config-1m.png`, `AA-C03-02-ttl-inicial.png`, `AA-C03-03-expirada.png`, `AA-C03-04-config-10m.png` y `AA-C03-05-ttl-restaurado.png`. Las capturas de TTL deben incluir la clave y la hora; no basta una imagen del número 60 sin contexto.

Resultado observado: Pendiente de tu ejecución. Registrar aquí cifras, IDs y lo que realmente ocurrió.

Estado: Pendiente. Cambiar a Conforme, No conforme o Evidencia incompleta al revisar las capturas.

Evidencias adjuntas: Anotar los nombres de los archivos que respaldan esta ficha.

C04 Crear una categoría con la lista previamente cacheada

Correspondencia: caso C-01 de la matriz del enunciado. **Lugar:** Postman y SPA.

1. Usa la marca de esta ronda y un nombre nuevo, por ejemplo `ZZ S13 AA ronda01 categoria`. No reutilices un nombre ya creado en otra prueba.
2. Calienta la lista con un GET de categorías ordenado por id descendente, tamanio=100 y soloActivas=true. Conserva exactamente esa URL para la consulta posterior. La segunda lectura previa ayuda a confirmar que ya había una copia reutilizable.
3. Captura la lista sin la categoría nueva. Crea la categoría desde la SPA o con POST /categorias, incluyendo nombre, descripción y estado=true. Guarda el ID devuelto.
4. Repite el mismo GET inmediatamente. Busca por el ID, no solo por texto. Debe aparecer aunque el TTL de 10 minutos todavía no haya vencido.
5. En la SPA comprueba la categoría. Si hay muchas filas, ordena ID descendente; no concluyas que falla porque una página distinta no la muestra.

Esperado: creación 201 y lista actualizada sin esperar TTL. **Capturas:** `AA-C04-01-lista-antes.png`, `AA-C04-02-creacion.png`, `AA-C04-03-lista-despues.png`. Deben acreditar que el GET posterior conserva los parámetros del anterior.

Resultado observado: Pendiente de tu ejecución. Registrar aquí cifras, IDs y lo que realmente ocurrió.

Estado: Pendiente. Cambiar a Conforme, No conforme o Evidencia incompleta al revisar las capturas.

Evidencias adjuntas: Anotar los nombres de los archivos que respaldan esta ficha.

C05 Renombrar una categoría y comprobar sus productos

Correspondencia: caso C-02 del enunciado. **Lugar:** Postman y Productos en la SPA.

1. Usa la categoría de ensayo que tiene un producto asociado. No renombres una categoría ajena a esta ronda.
2. Consulta dos veces la misma página de productos que contiene ese producto y captura su ID y categoriaNombre anterior. Así se comprueba una respuesta que ya pudo quedar en caché.
3. Edita el nombre de la categoría, por ejemplo de `ZZ S13 AA ronda01` a `ZZ S13 AA ronda01 nueva`. En Postman envía PUT /categorias/{id} con todos los campos requeridos, incluidos descripción y estado.
4. Repite inmediatamente el GET de productos con los mismos parámetros. La columna Categoria o categoriaNombre del mismo producto debe mostrar el nombre nuevo.
5. En la SPA recarga Productos normalmente. Para localizar el producto usa su categoría y selecciona 100 filas si está fuera de la primera página. Si no aparece en esos 100, localiza su página real; el filtro local no busca en todo el servidor.

Esperado: el nombre nuevo llega también al DTO de productos, no únicamente a /categorias. **Capturas:** `AA-C05-01-producto-antes.png`, `AA-C05-02-renombrado.png`, `AA-C05-03-producto-despues.png`. Si falla, revisar que update de categoría invalide categorias y productos; documentar defecto, cambio y repetición con ambas evidencias.

Resultado observado: Pendiente de tu ejecución. Registrar aquí cifras, IDs y lo que realmente ocurrió.

Estado: Pendiente. Cambiar a Conforme, No conforme o Evidencia incompleta al revisar las capturas.

Evidencias adjuntas: Anotar los nombres de los archivos que respaldan esta ficha.

C06 Editar un precio y comprobar detalle y lista

Correspondencia: caso C-03 del enunciado. **Lugar:** Postman y SPA.

1. Usa el producto de ensayo. Calienta tanto `GET /productos/{id}` como la página de lista que lo contiene, con dos lecturas idénticas de cada una.
2. Captura el precio anterior del mismo ID en detalle y lista. Cambia el precio, por ejemplo de 2,50 a 3,75. Conserva nombre, categoría, stock y estado para no introducir otras modificaciones involuntarias.
3. Guarda desde la SPA o envía PUT /productos/{id} con el objeto completo. Captura la respuesta correcta y el ID.
4. Repite el GET de detalle y el de lista inmediatamente, antes del TTL. Ambos deben contener 3,75, o el precio nuevo elegido en tu ronda.
5. Comprueba el producto en la SPA. Distingue el formato de moneda de la representación numérica del JSON.

Capturas: `AA-C06-01-detalle-antes.png`, `AA-C06-02-lista-antes.png`, `AA-C06-03-edicion.png`, `AA-C06-04-detalle-despues.png` y `AA-C06-05-lista-despues.png`. Una sola captura de la edición no demuestra que ambas cachés se invalidaron.

Resultado observado: Pendiente de tu ejecución. Registrar aquí cifras, IDs y lo que realmente ocurrió.

Estado: Pendiente. Cambiar a Conforme, No conforme o Evidencia incompleta al revisar las capturas.

Evidencias adjuntas: Anotar los nombres de los archivos que respaldan esta ficha.

C07 Venta desde la SPA y descuento de stock

Correspondencia: caso C-04 y evidencia de venta usada por D01. **Lugar:** SPA, Red y Postman.

1. Usa un producto de ensayo activo con stock conocido y un cliente ficticio activo. Anota ID, precio y stock. Si el producto tiene stock 10 y venderás 1, el esperado es 9.
2. Calienta la página de productos con dos GET idénticos. Captura el stock inicial y conserva sus parámetros.
3. Abre Nueva venta en la SPA con ADMIN o vendedor autorizado. Con Red abierto activa Preserve log y deja el filtro Fetch/XHR. Selecciona el cliente ficticio, agrega el producto y fija la cantidad.
4. Captura la revisión final de la venta con producto, cantidad y total, ocultando DNI. Desmarca imprimir si solo necesitas la evidencia en pantalla.
5. Confirma una sola vez y espera el comprobante. En Red identifica el POST /api/v1/ventas que respondió 201. Anota el ID de venta y su **X-Request-Id en Response Headers** antes de salir de la página. No confundas este identificador con el de un GET posterior.
6. Repite la misma consulta de productos del paso 2, sin esperar TTL. Captura el stock nuevo del mismo producto. Luego revísalo en la SPA, localizando la página que contiene el producto.
7. Calcula stock posterior = stock anterior − cantidad vendida. Conserva la venta en el historial; no la elimines para hacer que las cifras vuelvan a coincidir. Para repetir, toma el nuevo stock como siguiente línea base.

Capturas: `AA-C07-01-stock-antes.png`, `AA-C07-02-revision-venta.png`, `AA-C07-03-venta-201.png`, `AA-C07-04-stock-despues.png` y `AA-D01-01-request-id.png`. La última se reutiliza en D01. Es necesario probar que la venta se inició desde la SPA; un POST aislado de Postman no satisface esa parte de la consigna.

Resultado observado: Pendiente de tu ejecución. Registrar aquí cifras, IDs y lo que realmente ocurrió.

Estado: Pendiente. Cambiar a Conforme, No conforme o Evidencia incompleta al revisar las capturas.

Evidencias adjuntas: Anotar los nombres de los archivos que respaldan esta ficha.

C08 Baja de categoría vacía y consulta de inactivas

Correspondencia: caso C-05 del enunciado. **Lugar:** SPA y Postman.

1. Crea una categoría vacía exclusiva de esta ronda, sin productos. Anota su ID. No uses la categoría del producto vendido.
2. Calienta y captura una lista con `soloActivas=true` que contenga el ID. En la SPA deja **Mostrar inactivas desmarcado**. Si hace falta, ordena ID descendente para encontrar la categoría nueva.
3. Da de baja la categoría y captura el DELETE 204 o el mensaje de éxito. No debe fallar por productos asociados.
4. Repite inmediatamente la misma consulta activa: el ID ya no debe aparecer. Captura también la ausencia en la vista habitual de la SPA, manteniendo la búsqueda por el nombre de esta ronda.
5. Activa **Mostrar inactivas** y captura la misma categoría con estado Inactivo. En el API, la consulta sin soloActivas o con soloActivas=false conserva las bajas por diseño.
6. Explica el antes y después del defecto: antes la vista habitual mezclaba bajas; ahora filtra activas en el servidor antes de paginar y separa las variantes en la clave de caché. No se ha cambiado la baja lógica por borrado físico.

Capturas: `AA-C08-01-activa-antes.png`, `AA-C08-02-baja.png`, `AA-C08-03-activa-despues.png` y `AA-C08-04-inactivas.png`. No marques fallo solo porque la consulta administrativa sigue mostrando estado=false; el criterio de ausencia corresponde a la lista activa.

Resultado observado: Pendiente de tu ejecución. Registrar aquí cifras, IDs y lo que realmente ocurrió.

Estado: Pendiente. Cambiar a Conforme, No conforme o Evidencia incompleta al revisar las capturas.

Evidencias adjuntas: Anotar los nombres de los archivos que respaldan esta ficha.

C09 Redis detenido y cuatro respuestas de health

Correspondencia: caso C-06 y monitoreo de health de la parte D. **Lugar:** terminal FRONTEND, SPA y Postman.

1. Con Redis encendido, consulta health con **No Auth**: captura HTTP y cuerpo sin componentes. En otra petición con ADMIN consulta el mismo health y captura los componentes, incluido redis UP.

2. Mantén Oracle y el backend funcionando. Desde FRONTEND ejecuta solo:

```
docker compose -f deploy/docker-compose.yml stop redis
```

3. No detengas web, Oracle ni toda la composición. Espera la parada y captura su estado en Docker.

4. En la SPA abre Productos y recarga normalmente. Acredita una petición real del API con HTTP 200 y su cuerpo; el contenido anterior que permanece dibujado no prueba continuidad. También puede comprobarse Categorías, pero la matriz exige Productos.

5. Consulta health sin token: se espera HTTP 503 y estado DOWN sin detalles. Consulta con ADMIN: se espera 503/DOWN con el componente Redis fallido. Captura las dos respuestas, ocultando Authorization.

6. Busca una advertencia de caché en el log de esta petición. La operación sigue con Oracle; puede tardar más por los timeouts. No la interpretes como un error que obliga a detener la prueba.

7. Al terminar, incluso si algo falla, restaura Redis:

```
docker compose -f deploy/docker-compose.yml start redis
```

8. Comprueba ping de Redis, health UP y un GET de productos 200. No crees ventas ni edites catálogos durante esta prueba: una invalidación perdida durante la caída requiere tratamiento adicional antes de confiar en claves recuperadas.

Capturas: `AA-C09-01-health-up-publico.png`, `AA-C09-02-health-up-admin.png`, `AA-C09-03-redis-parado.png`, `AA-C09-04-productos-200.png`, `AA-C09-05-health-down-publico.png`, `AA-C09-06-health-down-admin.png`, `AA-C09-07-warn.png` y `AA-C09-08-restaurado.png`.

Registrar en una tabla: Redis ON/OFF × público/ADMIN, HTTP, status global y presencia de detalles. En recuperación anota UP. Health global DOWN por Redis no demuestra que el proceso Java o Oracle hayan caído.

Resultado observado: Pendiente de tu ejecución. Registrar aquí cifras, IDs y lo que realmente ocurrió.

Estado: Pendiente. Cambiar a Conforme, No conforme o Evidencia incompleta al revisar las capturas.

Evidencias adjuntas: Anotar los nombres de los archivos que respaldan esta ficha.

D01 Reconstruir la venta en el log

Requisito: parte D, seguir una venta desde su identificador de Red. **Lugar:** la captura de C07 y terminal BACKEND.

1. Usa el X-Request-Id del POST de la venta de C07, junto con fecha y hora. No vuelvas a vender solo para obtener una captura que ya tienes.

2. En BACKEND filtra el archivo real de producción, sustituyendo el ejemplo por ese ID:

```
Select-String -LiteralPath logs/pharma-backend.log -SimpleMatch 'ID-DE-TU-POST'
```

3. Si el proyecto tiene rg instalado puedes usar `rg 'ID-DE-TU-POST' logs/pharma-backend.log`. La salida debe mostrar todas las líneas de esa petición, no solo una selección favorable.

4. Captura las líneas completas y legibles. En esta implementación se espera inicio de registro, venta persistida y cierre del POST con estado y duración. Si tu salida difiere, explícalo según las líneas reales.

5. Relaciona el ID de venta del log con el comprobante. Explica qué cuenta cada línea y que el evento interno «persistida» se escribe antes de terminar la transacción; la respuesta 201 completa la evidencia del éxito HTTP.

Capturas: reutiliza `AA-D01-01-request-id.png` de C07 y añade `AA-D01-02-log-venta.png`. Deben tener el mismo ID de petición. El requestId es por petición, no por toda la sesión.

Resultado observado: Pendiente de tu ejecución. Registrar aquí cifras, IDs y lo que realmente ocurrió.

Estado: Pendiente. Cambiar a Conforme, No conforme o Evidencia incompleta al revisar las capturas.

Evidencias adjuntas: Anotar los nombres de los archivos que respaldan esta ficha.

D02 Código de seguimiento en el backend y en la SPA

Requisito: parte D, requestId en ErrorResponseDTO y en los mensajes de error. **Lugar:** Postman, SPA, Red y log.

1. Con ADMIN pide un producto cuyo ID hayas verificado como inexistente, por ejemplo un número alto de prueba. No edites un producto real para provocar el error.
2. En el 404 de `GET /productos/{id}` captura el cuerpo con requestId, status, message y path. En Response Headers captura X-Request-Id y confirma que ambos valores coinciden para esa misma petición.
3. Abre la edición de ese ID desde la SPA, `/productos/{id}/editar`. Captura el mensaje «Código de seguimiento: ...». Selecciona en Red el GET fallido originado por esta navegación y compara ese requestId con el de la pantalla. El GET de Postman del paso 2 es otra petición y puede tener un ID diferente.
4. Busca en el log el ID de la petición de la SPA y captura la línea. El usuario puede comunicar ese código y la hora al soporte sin compartir sus credenciales.
5. Como comprobaciones adicionales, prueba un GET protegido con No Auth (401), una categoría con nombre vacío (400, sin crear datos), y un intento de venta con stock insuficiente (409, usando solo el producto ficticio). En cada respuesta verifica requestId y su cabecera. No provoques un 500 dañando Oracle: el saneamiento de fallos inesperados se valida con la prueba automatizada correspondiente.

Capturas obligatorias: `AA-D02-01-error-cuerpo.png`, `AA-D02-02-error-cabecera.png`, `AA-D02-03-error-spa.png` y `AA-D02-04-log.png`. **Complementarias:** `AA-D02-05-401.png`, `AA-D02-06-400.png` y `AA-D02-07-409.png`. Si una respuesta carece del campo, registra el defecto en vez de copiarle el ID de otra petición.

Resultado observado: Pendiente de tu ejecución. Registrar aquí cifras, IDs y lo que realmente ocurrió.

Estado: Pendiente. Cambiar a Conforme, No conforme o Evidencia incompleta al revisar las capturas.

Evidencias adjuntas: Anotar los nombres de los archivos que respaldan esta ficha.

D03 Auditoría de secretos en el log de producción

Requisito: parte D, revisar contraseñas, tokens, clave JWT y DNI completos. **Lugar:** terminal BACKEND y revisión del código de logs.

1. Comprueba que `logs/pharma-backend.log` existe y corresponde a la ronda en perfil prod. Captura ruta, tamaño y última modificación. Un archivo ausente o vacío no acredita una auditoría satisfactoria.

2. Ejecuta la búsqueda de palabras del enunciado sin imprimir las líneas coincidentes. Mostrar su cantidad permite acreditar la revisión sin exponer una posible contraseña en la captura.

```
Get-Item -LiteralPath logs/pharma-backend.log | Select-Object Name,Length,LastWriteTime
$aaCoincidencias = Select-String -LiteralPath logs/pharma-backend.log -Pattern 'password|Bearer |secret'
'Coincidencias de palabras: ' + @($aaCoincidencias).Count
```

3. Añade búsquedas de JWT con tres segmentos y de DNI explícito. Para la clave JWT real y las contraseñas usadas, comprueba coincidencia literal de los valores solo en memoria y muestra únicamente sí/no o conteo, nunca esos valores en la terminal. El ayudante del anexo 2 evita escribirlos en el historial.

4. Una coincidencia de la palabra «password» puede ser un nombre de clase; revísala localmente antes de concluir que hay una fuga. Una búsqueda sin coincidencias no demuestra por sí sola que ningún secreto pueda existir con otro formato.

5. Revisa los puntos que escriben el log: no deben concatenar el cuerpo del login, Authorization, secreto de conexión ni DNI completo. En ClienteServiceImpl se retiró el DNI del evento de registro; anota el cambio si lo usas como hallazgo.

6. Si hay una fuga, guarda solo evidencia redactada de la causa, corrige el log y repite la operación en un archivo o periodo nuevo. Documenta los resultados antes/después. Corregir el código no borra un log histórico; no alteres un original para afirmar que nunca hubo coincidencias.

Capturas: `AA-D03-01-archivo-y-conteos.png`, `AA-D03-02-patrones.png` y, si hubo hallazgo, `AA-D03-03-correccion.png` y `AA-D03-04-repeticion.png`. **Registrar:** archivo/periodo auditado, patrones, conteos, revisión de falsos positivos y resultado de la repetición.

Resultado observado: Pendiente de tu ejecución. Registrar aquí cifras, IDs y lo que realmente ocurrió.

Estado: Pendiente. Cambiar a Conforme, No conforme o Evidencia incompleta al revisar las capturas.

Evidencias adjuntas: Anotar los nombres de los archivos que respaldan esta ficha.

D04 Métricas HTTP de ventas

Requisito: parte D, consultar e interpretar http.server.requests con uri de ventas. **Lugar:** Postman con ADMIN.

1. Con la misma JVM de la venta C07 consulta la siguiente URL. Si ya reiniciaste por otra prueba, ese contador anterior no se conserva: registra el nuevo periodo y ejecuta una nueva operación de ensayo si necesitas una muestra del nuevo arranque.

```
GET {{backendUrl}}/actuator/metrics/http.server.requests?tag=uri:/api/v1/ventas
```

2. Captura nombre, baseUnit, measurements y availableTags. La consulta de la métrica no cuenta como otra petición a /api/v1/ventas.

3. Interpreta COUNT como número de solicitudes observadas con ese filtro y TOTAL_TIME como suma en segundos. Media = TOTAL_TIME / COUNT si COUNT es mayor que cero. Multiplica por 1000 para expresarla en ms. MAX no es el percentil 95 y puede depender de la ventana del medidor.

4. Si aparecen varios métodos o estados, no atribuyas todos a la venta del formulario. Para aislar el registro de ventas, añade `&tag=method:POST`; puedes añadir `&tag=status:201` si esa combinación existe. Conserva también la consulta por uri solicitada por la actividad.

5. Si la métrica o combinación devuelve 404, comprueba autenticación y que esa JVM ya atendió una petición con esa ruta y etiquetas. No copies métricas de /ventas/buscar: es otra uri.

Capturas: `AA-D04-01-uri-ventas.png` y `AA-D04-02-post-ventas.png` si se necesita desagregar. Incluye URL, estado 200, COUNT, TOTAL_TIME y MAX. **Registrar:** periodo desde el arranque, filtros, media calculada y límites de una muestra pequeña.

Resultado observado: Pendiente de tu ejecución. Registrar aquí cifras, IDs y lo que realmente ocurrió.

Estado: Pendiente. Cambiar a Conforme, No conforme o Evidencia incompleta al revisar las capturas.

Evidencias adjuntas: Anotar los nombres de los archivos que respaldan esta ficha.

E01 Revisión del despliegue y de los dos README

Requisito: parte E, dejar una sección de producción en cada repositorio. **Lugar:** README de BACKEND y FRONTEND, código de configuración y terminales.

1. Abre «Despliegue en producción» en ambos README. Verifica que el diagrama identifica navegador, Nginx, PharmaBackend, Oracle y Redis. Si el editor muestra Mermaid como texto, abre su vista previa renderizada para capturar el diagrama.
2. Distingue puertos de laboratorio y propuesta: Nginx host 8081/contenedor 80; backend 8080; Oracle local host 1522/contenedor 1521; Redis 6379. HTTPS 443 es una recomendación de producción si aún no se instaló el certificado; no lo presentes como probado.
3. Verifica que las variables tienen propósito y ningún valor real. Revisa perfil prod, DB, JWT, Redis, CACHE_TTL, orígenes y puertos. Angular no debe recibir secretos del backend.
4. Comprueba que existen pasos de construcción, arranque, controles de salud y reversión. En la reversión se requiere conservar JAR, frontend y configuración compatibles; volver al JAR anterior no revierte una migración de Oracle.
5. Completa la lista de prácticas con Cumple, Parcial o No cumple y acción pendiente. No conviertas una recomendación, por ejemplo backups restaurables, en un cumplimiento sin prueba.
6. Redacta tres prioridades justificadas para los tres locales: HTTPS y acceso privado; backups Oracle con restauración ensayada; Redis compartido con alertas y procedimiento de recuperación. Anota que las ventas actuales requieren conexión.

Capturas o evidencias: `AA-E01-01-readme-backend.png`, `AA-E01-02-readme-frontend.png` y `AA-E01-03-arquitectura.png`. Entregar los archivos README es obligatorio; sus capturas son apoyo y no los sustituyen. No hace falta desplegar en servidores de los tres locales para acreditar esta parte de recomendaciones.

Resultado observado: Pendiente de tu ejecución. Registrar aquí cifras, IDs y lo que realmente ocurrió.

Estado: Pendiente. Cambiar a Conforme, No conforme o Evidencia incompleta al revisar las capturas.

Evidencias adjuntas: Anotar los nombres de los archivos que respaldan esta ficha.

F01 Pruebas automatizadas como complemento

Alcance: validación adicional del código. No reemplaza las partes A a D. **Lugar:** terminal de cada proyecto.

1. En FRONTEND ejecuta `npm.cmd test -- --watch=false`. Captura el resumen final y conserva cualquier fallo que aparezca.
2. En BACKEND, con Java 21 y dependencias disponibles, ejecuta las pruebas aisladas. En PowerShell el argumento entre comillas evita interpretar el signo de exclusión:

```
.\mvnw.cmd test '-Dtest=!PharmaBackendApplicationTests'
```

3. Esta selección excluye la prueba de contexto que depende del entorno externo. Si ejecutas toda la suite, documenta su configuración y su resultado por separado; no presentes la exclusión como una aprobación del contexto completo.
4. Para construir el JAR con las pruebas correspondientes, detén antes la instancia que ejecuta ese mismo JAR y ejecuta package con la misma selección. Reinicia el servicio y comprueba health después.
5. Registra los conteos actuales. La ronda anterior tuvo 53 pruebas de frontend y 42 de backend; una cifra distinta no es fallo por sí misma si el código y los casos cambiaron.

Capturas: `AA-F01-01-tests-frontend.png` y `AA-F01-02-tests-backend.png`, con comando, tests, fallos, errores y final correcto. **Registrar:** qué se excluyó y por qué, sin sustituir las capturas manuales de la actividad por estos dos resúmenes.

Resultado observado: Pendiente de tu ejecución. Registrar aquí cifras, IDs y lo que realmente ocurrió.

Estado: Pendiente. Cambiar a Conforme, No conforme o Evidencia incompleta al revisar las capturas.

Evidencias adjuntas: Anotar los nombres de los archivos que respaldan esta ficha.

F02 Versionamiento y entrega

Requisito: parte F, tres commits en días distintos por repositorio, PDF y solicitudes a develop. **Lugar:** cada repositorio, GitHub y aula virtual.

1. Revisa `git status --short` en cada proyecto. Incluye solamente cambios correspondientes a la actividad; no añadas automáticamente archivos con credenciales ni documentos de otras sesiones.
2. Revisa el historial de la rama con el comando siguiente. Cuenta los commits de trabajo de esta actividad en fechas reales, no cualquier commit antiguo de otra sesión.

```
git log --date=iso-local --format='%h %ad %s' -n 12
```

3. Completa el requisito de tres días distintos con avances reales. Usa mensajes de la convención perf, feat, fix, docs o chore. No cambies fechas para simular jornadas. En la entrega anterior se guardó un commit local por repositorio, pero eso solo no acredita tres días.
4. Revisa el informe: nombre completo, partes A a E, tablas, capturas numeradas junto a cada caso, resultados realmente observados y explicación de hallazgos corregidos. Completa la matriz de seis casos con ID oficial C-01 a C-06.
5. Guarda el informe final en BACKEND como `docs/informe-s13.pdf` y prepara la copia `Pacho_LP2_S13_Autonoma.pdf` para el aula. Conserva su DOCX editable con el diseño de las sesiones anteriores. Esta guía de pasos no sustituye ese informe final.
6. Publica las ramas revisadas y abre una solicitud hacia `develop` en cada repositorio. Captura título, rama origen, rama destino, estado y enlace real. Un texto preparado para PR no equivale a una solicitud creada.
7. Sube la copia del PDF al aula virtual y conserva su comprobante. La consigna fija el plazo del 11 de noviembre de 2026 a las 23:59, hora de Lima. No marques entregado antes de confirmar la subida.

Capturas: `AA-F02-01-historial-backend.png`, `AA-F02-02-historial-frontend.png`, `AA-F02-03-pr-backend.png`, `AA-F02-04-pr-frontend.png` y `AA-F02-05-aula.png`. Antes de publicar, revisa que ninguna imagen o archivo del informe contenga secretos completos.

Resultado observado: Pendiente de tu ejecución. Registrar aquí cifras, IDs y lo que realmente ocurrió.

Estado: Pendiente. Cambiar a Conforme, No conforme o Evidencia incompleta al revisar las capturas.

Evidencias adjuntas: Anotar los nombres de los archivos que respaldan esta ficha.

Matriz que debe quedar en el informe

ID del enunciado	Ficha de esta guía	Qué se compara	Evidencia que no puede faltar
C-01	C04	Categoría ausente → creada y visible	Lista cacheada, 201 y misma lista actualizada
C-02	C05	Nombre de categoría en un producto	Producto antes, edición y producto con nombre nuevo
C-03	C06	Precio en detalle y lista	Ambas lecturas antes y después
C-04	C07	Stock anterior − cantidad vendida	Stock inicial, venta desde SPA y stock posterior
C-05	C08	Presencia en activas tras baja	Baja, ausencia en activas y consulta de inactivas
C-06	C09	Funcionamiento sin Redis y health	GET real 200, DOWN y recuperación UP

Campos para completar por cada ficha: fecha y hora; versión o commit; herramienta y configuración; datos/IDs de ensayo; esperado; observado; medición antes; medición después; archivos de captura; estado; interpretación y, si falla, corrección y repetición. Deja Pendiente lo que aún no ejecutaste.

Anexo 1 Ayuda para los tiempos de navegación

En A03, pega este código en Console mientras estás en `/login`, antes de pulsar Ingresar. Si Chrome bloquea pegar, lee el aviso del navegador; no desactives protecciones del sistema. También puedes guardar el código como Snippet en DevTools. No contiene ni lee contraseña, token o datos del formulario.

```
(() => {
  const reloj = { login: null, ventas: null, terminado: false };
  document.addEventListener('click', e => {
    const elemento = e.target.closest('button,a');
    if (!elemento) return;
    if (elemento.tagName === 'BUTTON' && elemento.textContent.trim() === 'Ingresar') {
      reloj.login = performance.now();
    }
    if (elemento.tagName === 'A' && elemento.getAttribute('href') === '/ventas') {
      reloj.ventas = performance.now();
    }
  }, true);
  const observador = new MutationObserver(() => {
    const titulo = document.querySelector('h2');
    if (reloj.terminado || reloj.login === null || reloj.ventas === null ||
        location.pathname !== '/ventas' || titulo?.textContent.trim() !== 'Ventas') return;
    reloj.terminado = true;
    requestAnimationFrame(() => requestAnimationFrame(() => {
      const fin = performance.now();
      console.table({
        loginAVentasMs: +(fin - reloj.login).toFixed(1),
        clicAVentasMs: +(fin - reloj.ventas).toFixed(1)
      });
      observador.disconnect();
    }));
  });
  observador.observe(document.body, { childList: true, subtree: true });
  console.info('Medición preparada para el siguiente login y clic en Ventas.');
})();
```

Pulsa el botón Ingresar con el mouse, espera los 5 segundos fijados y abre Ventas con el menú. Captura la tabla. El criterio es el encabezado visible tras dos cuadros de renderizado; no es una medida del tiempo total del API. Cada nueva ventana requiere volver a preparar el ayudante. Usa el mismo criterio y herramienta en ambas variantes.

Anexo 2 Ayuda PowerShell sin imprimir secretos

Para las métricas puedes seguir usando Postman. Si necesitas curl autenticado sin mostrar el token en el comando de la captura, abre una terminal privada en FRONTEND e introduce el token de tu login como dato oculto. No lo escribas directamente en el historial. Este código no inicia sesión ni guarda credenciales en archivos.

```
$aaTokenSeguro = Read-Host 'Pega el token ADMIN sin Bearer' -AsSecureString
$aaTokenLocal = [System.Net.NetworkCredential]::new('', $aaTokenSeguro).Password
$aaConfigCurl = 'header = "Authorization: Bearer ' + $aaTokenLocal + '"'
'Recurso autenticado: http://localhost:8081/api/v1/categorias'
$aaConfigCurl | curl.exe -sS -I -K - http://localhost:8081/api/v1/categorias
Remove-Variable aaTokenLocal,aaTokenSeguro,aaConfigCurl
```

Para buscar un secreto concreto en el log sin imprimirlo ni mostrar coincidencias completas, abre una terminal BACKEND y repite este bloque por cada contraseña de ensayo o clave que necesites auditar. El resultado solo indica cantidad. No captures la administración de variables donde el secreto quede visible.

```
$aaSecretoSeguro = Read-Host 'Secreto que comprobarás en el log' -AsSecureString
$aaSecretoLocal = [System.Net.NetworkCredential]::new('', $aaSecretoSeguro).Password
if ([string]::IsNullOrWhiteSpace($aaSecretoLocal)) { throw 'Introduce un valor no vacío.' }
$aaNumero = @(Select-String -LiteralPath logs/pharma-backend.log -SimpleMatch -Pattern $aaSecretoLocal).Count
'Coincidencias literales del secreto revisado: ' + $aaNumero
Remove-Variable aaSecretoLocal,aaSecretoSeguro
```

Para los patrones adicionales de D03, muestra solo el conteo:

```
$aaPatronJwt = '\beyJ[A-Za-z0-9_-]+\.[A-Za-z0-9_-]+\.[A-Za-z0-9_-]+'
$aaPatronDni = '(?i)DNI\s*[=:]\s*\d{8}\b'
'JWT completos: ' + @(Select-String -LiteralPath logs/pharma-backend.log -Pattern $aaPatronJwt).Count
'DNI explícitos completos: ' + @(Select-String -LiteralPath logs/pharma-backend.log -Pattern
$aaPatronDni).Count
```

Referencias de procedimiento

La cobertura de pruebas sigue las partes A a F del PDF de actividad autónoma S13 proporcionado por el estudiante. La organización de fichas retoma los documentos editables locales de S10, S11 y la práctica S13. Se revisaron las rutas, claves de caché, puertos, perfil prod y opciones de la versión local para escribir estos pasos.

• Angular Build: https://angular.dev/tools/cli/build

• Angular PreloadAllModules: https://angular.dev/api/router/PreloadAllModules

• esbuild Bundle Size Analyzer: https://esbuild.github.io/analyze/

• Spring Boot Metrics: https://docs.spring.io/spring-boot/reference/actuator/metrics.html

• Redis TTL: https://redis.io/docs/latest/commands/ttl/

• MDN Cache-Control: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Cache-Control

La documentación respalda los conceptos; las capturas y resultados del informe deben proceder de tu ejecución. Conserva también los errores y las repeticiones, identificados con su fecha.